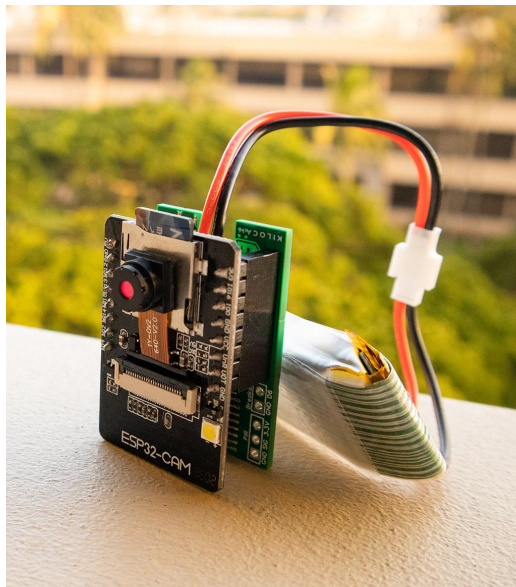


# KiloCam Datasheet & FAQ

A low-cost, low-power habitat monitoring camera

## KiloCam Overview (FAQ at end of document)

KiloCam is a printed-circuit board (PCB) designed to attach to the back of commercially available ESP32-Cam boards. On their own, ESP32-Cam boards (black PCB in photo) are cheap and include a 2MP camera, but are not power-efficient. KiloCam (Green PCB in photo) is



designed to deliver power to the ESP32-Cam board only at user-specified time periods for photo capture - this allows improved battery life and extended deployment times. Images captured are stored to a MicroSD card on the ESP32-Cam for easy data retrieval.

**Cost:** The ESP32-Cam PCB (not provided) is typically \$5-8 USD online. The KiloCam board is \$20-25 USD depending on availability.

**Image quality:** The ESP32-Cam, on which KiloCam is based, utilizes a 2MP OV2640 camera with a 1/4" sensor. While low-resolution and not suitable for low-light environments, this camera is sufficient for

many habitat monitoring applications (see "KiloCam Test Images").

**Assembly:** The complete camera module (KiloCam + ESP32-Cam) is assembled by snapping the two boards together with the included header pins, no soldering is needed. A power source of your choice (3.3 to 8 V) is attached via the included two-pin screw terminals. **Based on early tests a KiloCam capturing 1 photo per minute lasts ~60 hours on a 750 mAh capacity battery. For reference, a rechargeable AA battery has a capacity of 2200 mAh.** A complete build-guide video is now available at this link: <https://www.youtube.com/watch?v=vf54ca9luP4&t=21s>

**Programming:** All programming is done using the Arduino IDE and a cheap USB-Serial FTDI programmer. The ESP32-Cam PCB only needs to be programmed once using the provided code to set image quality, contrast, white balance etc. The KiloCam PCB is programmed once to set the real-time clock, and once to define the capture start time and the sampling interval. You may reprogram the KiloCam PCB at any time to change these settings.

**Other notes:** The KiloCam PCB includes pinouts for a Proximity Infrared (PIR) motion sensor (untested) and a digital input/output pin to trigger externally-powered lights or motors with the addition of a cheap (\$3 USD) relay board. KiloCam is designed so it and a 750 mAh LiPo battery fit into GoPro Hero 3/4/5 waterproof housings for field deployments.

## KiloCam Test Images

No color corrections or editing have been performed on these handheld test images.

Image of *Porites* coral colony taken at 15 m with damselfish, about 3 meters away.





Image of Hibiscus flower in shade, taken from < 1 meter away.



Image through a window of building (200 m away) and hillside (1 km away) in full sunlight





Image of black feral cat in shade under bushes (note tendency of camera to prioritize shadow exposure, leading to periodic overexposure of especially bright areas in full sunlight)





Image of walkway with complex tree canopy





Image of tree, highlighting details captured from approximately 2 m away





Comparison of KiloCam imagery with stock lens (LEFT) and Kilocam adapted to use 6mm M12 lens without infrared cutoff filter (RIGHT). Adapted KiloCam photos may be slightly out of focus.

### Building



### Small finches on grass



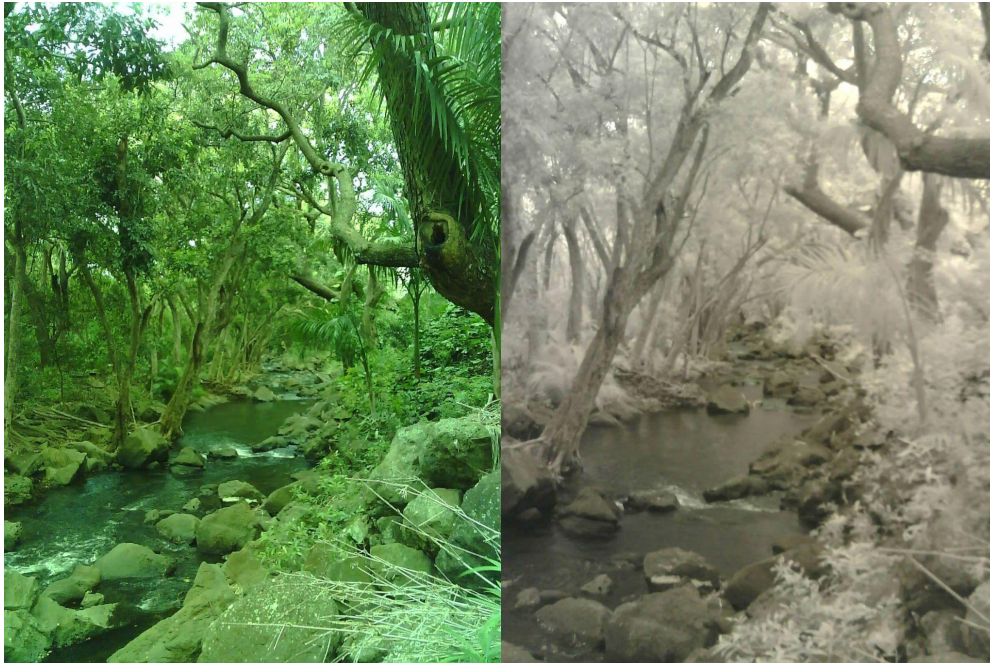


## Duck in water

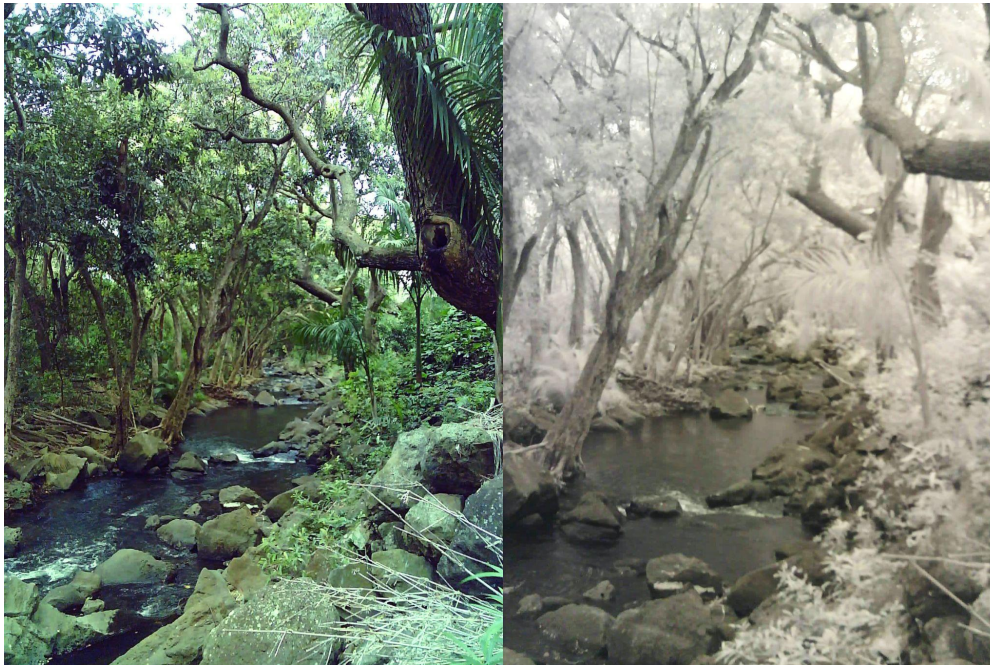




## Stream



Stream images after color correction, contrast, and sharpening





Bird on grass



## Pathway under trees





## KiloCam FAQ

**Q: What are the materials needed to build and use KiloCam?**

A: Here is a complete table of materials needed to use KiloCam

### Materials List

|   | Item                | Quantity   | Cost        | Purpose                                                                                                                                                                    | Links                                                                                                            |
|---|---------------------|------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| x | KiloCam PCB         | 1 per unit | \$20-25     | Timed data capture                                                                                                                                                         |                                                                                                                  |
| x | ESP32-Cam PCB       | 1 per unit | \$5-8       | Camera functionality                                                                                                                                                       | <a href="#">Amazon</a> , <a href="#">Ebay</a>                                                                    |
| x | MicroSD card        | 1 per unit | \$5         | Image storage                                                                                                                                                              | <a href="#">Amazon</a> , or any 16GB                                                                             |
| x | CR1220 battery      | 1 per unit | \$3         | RTC backup battery                                                                                                                                                         | <a href="#">Amazon</a>                                                                                           |
| x | 3.3 to 8 V battery  | 1 per unit | N/A, varies | Power source                                                                                                                                                               | <a href="#">These LiPos work well</a> . Be aware of LiPo usage/charging concerns. Any battery 3.3-8 V will work. |
| x | Battery connectors  | 1 per unit | N/A, varies | Connect battery to KiloCam screw terminals. Make sure your connector type matches the connector on your battery. Bare wire ends are inserted into KiloCam screw terminals. | <a href="#">These connectors work with the LiPo battery linked above</a> .                                       |
| x | USB FTDI Programmer | 1 total    | \$6         | Programming                                                                                                                                                                | <a href="#">Amazon</a>                                                                                           |

|   |                             |         |     | device                                                                                                                                    |                                                                              |
|---|-----------------------------|---------|-----|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| x | Female-Female jumper cables | 5 total | \$1 | For programming ESP32-Cam, as needed.                                                                                                     | <a href="#">Amazon (set of 60)</a>                                           |
|   | Laptop                      | 1 total | N/A | Programming device. <a href="#">Install the Arduino IDE</a> and ensure it operates correctly. Runs best on Windows PCs, but Mac can work. | <a href="#">If you don't have a Windows PC, this Asus laptop works well.</a> |

**Q: Are there instructions for using KiloCam?**

A: I am in the process of making instructional videos, or a user guide, and will add them to this document when complete. This will occur before units are mailed for beta testing.

**Q: Can I attach a higher resolution camera to KiloCam?**

A: In short, no. The ESP32-Cam board used by KiloCam does not have appropriate drivers for higher resolution camera modules. If you need higher resolution photos or videos, can solder, and don't need a high capture frequency [check out my video on CoralCam](#) or the [HardwareX publication](#). CoralCam's primary limitation is the internal camera battery. While higher resolution than KiloCam, a deployment of CoralCam at 3 photos per day will last approximately 2 months, and a deployment at 1 photo per hour will last approximately 12-14 days.

**Q: Can KiloCam take photos at night?**

A: In the KiloCam code you can define any range of hours in which KiloCam will capture photos should your sampling interval occur (example: only between 06:00 and 18:00). However, KiloCam is not a night-vision camera and these images will be black. I recommend setting the capture range to fall within daylight hours.

As mentioned above in "Other notes" the KiloCam PCB does have the ability to trigger an externally-powered light source on or off using a cheap relay board and the included digital input/output pin. If this is something you want to do just email me and I can walk you through



the extra steps. I would highly suggest that your light source be outside the visible range of the organisms you are trying to observe.

**Q: Can KiloCam take videos?**

A: Technically yes, but I have yet to explore this capability as it will be at lower resolution than the photos, will have a relatively low framerate, and will likely deplete power sources faster.

**Q: Is KiloCam waterproof?**

A: It is if you put it inside a waterproof housing, such as a GoPro Hero 3,4,5 housing. A “naked” KiloCam should never be deployed in an outdoor environment.

**Q: Does KiloCam have a “real-time clock”?**

A: Yes, I designed the KiloCam board to use a DS3231SN or DS3231M real-time clock. The KiloCam PCB also includes a backup battery holder that uses a CR1220 coin-cell battery. This ensures that even when the KiloCam PCB does not have a power source the real-time clock can continue keeping track of time. If it didn't have this, you would need to reset the clock each time you removed the power source. Including a real-time clock also allows you to set up multiple KiloCam units to capture data at nearly the same time. Please note that I use the DS3231SN and DS3231M interchangeably depending on availability. The DS3231SN has an estimated drift of 3 seconds per month, while the DS3231M has an estimated drift of about 10 seconds per month.

**Q: Is condensation inside the GoPro housings a problem?**

A: Typically not, but I always recommend including a small desiccant packet inside the GoPro housing just in case.

**Q: How long will KiloCam run for?**

A: Run time depends on your duty cycle (how often you take a photo) and the capacity of the battery used. KiloCam will accept most power sources that are between 3.3 and 8 V. Early tests showed KiloCam will run for 2.5 days at one photo per minute off of a 750 mAh battery. Napkin calculations indicate the same battery should provide approximately 80-100 days of operation at one photo per hour. For reference, the average rechargeable AA battery has a capacity of 2200-2400 mAh. As KiloCam is new, I have not been able to run a year-long battery test.

**Q: How close can KiloCam focus?**

A: The camera on the ESP32-Cam PCB generally has a factory-set near focus limit of about 50

cm. You should be able to adjust this limit to be closer by twisting the OV2640 lens gently. I have also had success 3D-printing an adapter so many 12mm (M12) lenses common to GoPro and security cameras can be mounted. These lenses provide a wide variety of potential focus distances and fields of view. **The default ESP32-Cam lens has a focal length of ~60 mm.**

**Q: How quickly can KiloCam take photos?**

A: Currently I would not recommend a photo interval shorter than 30 seconds. The shorter the interval gets between data captures the more power is consumed, shortening deployment time. If you are able to provide constant power and would just like to use the KiloCam PCB as an intervalometer then I would not recommend intervals shorter than 5-10 seconds.

**Q: Why do the test images have a green hue?**

A: The test images have had no color corrections applied, so this hue can be removed. In addition, while programming the ESP32-Cam you can specify a particular white balance, contrast, and other basic settings to improve image-to-image consistency.

**Q: How many images can KiloCam take?**

A: Online reports list that the ESP32-Cam board used by KiloCam should support up to 16GB of images. At a maximum of 400 kB per image, that is roughly 40,000 images. The microSD card and your computer are likely to encounter trouble parsing through that many files, so I would plan your deployments to ideally not exceed 10,000 images.

**Q: How big is KiloCam?**

A: The KiloCam PCB, when attached to the ESP32-Cam, measures approximately 40x25x25 mm.

**Q: Are you an electrical engineer?**

A: No. I am an ecologist who likes to tinker. With that in mind, there are certainly better ways of designing KiloCam and there might be gremlins lurking somewhere in the code I've written to operate it. Please be patient :)

**Q: If I buy a KiloCam PCB, where does my money go?**

A: I manufacture the KiloCam PCBs overseas in small batches as demand dictates and finish/test them in Hawaii. Any proceeds from KiloCam (or CoralCam) PCBs are used to fund development of future low-cost instruments under my business (Ecologis Consulting LLC) registered in the state of Hawaii. I do not make a living from these sales, the business only



exists to handle taxes and pay for manufacturing. I volunteer my time to develop these instruments in the hope of making science more accessible to those who lack funding.

Q: Can I remove a SD card from Kilocam, read the card from my computer, and put it back to the Kilocam during a sleep period?

A: Yes you can! The ESP32-Cam has no power whatsoever while the KiloCam PCB is in a sleep period so removal of the sd card will not lead to any disk-mounting issues. I do this all the time to make sure cameras are operating as expected.

**Q: Can I replace a battery during a sleep period? I think this will reset Kilocam. Then, say that I programmed Kilocam to start at 8 am. Several days later, I replaced the battery at 11 am. Will Kilocam start again at 8 am the next day?**

A: Removing power from the KiloCam PCB will reset the KiloCam, as you suspected. You are correct in your understanding of the proposed scenario. If you programmed KiloCam to have an initial wakeup time of 08:00 and disconnected the power source at any point during operation then it will restart data collection when it gets power again. If you reconnected power before 08:00, then it would start collecting again that same day at 08:00. If you reconnected power at 08:01 or later then it would restart collections the next day at 08:00 and continue at the programmed interval and dawn-dusk range.

**Q: Does the wake-up time (say 8 am) is the time Kilocam takes the first picture of the day or 30 min later (8:30 am)?**

A: You set four values in the KiloCam PCB code: the sampling interval, Dawn, Dusk, and the initial wakeup time (see build guide video). The first photo will always occur at the programmed initial wakeup time, at which point photos will be collected at the provided interval until the last possible interval within the Dusk hour value is reached (if one is specified). Collections will then continue the next morning at the earliest possible interval within the Dawn hour. I generally recommend that your initial wakeup time and your Dawn values should be the same, but you can offset the two to achieve creative sampling designs. That said, I understand that anything other than a 24 hour cycle can lead to some headache when assigning timestamp values to photos post-hoc. To remedy that I have written a new ESP32-Cam code file that will name images with a projected time of capture based on the user-specified interval, dawn, dusk, initial wakeup value, and number of photos taken - I'm testing this now and will release it to users as soon as it seems reliable.

- **Example 1 - initial wakeup and dawn do not match:** Interval is 1800 seconds (30 min), Dawn is 08, Dusk is 20, initial wakeup is 06:00. Board is provided power at 3pm. This board will take photos every 30 minutes the next morning beginning at 06:00 until 19:30. The next day and thereafter it will take photos every 30 minutes from 08:00 to 19:30.
- **Example 2 - initial wakeup and dawn do match:** Interval is 1800 seconds (30 min), Dawn is 08, Dusk is 20, initial wakeup is 08:00. Board is provided power at 3pm. This board will take photos every 30 minutes the next morning beginning at 08:00 until 19:30. The next day and thereafter it will take photos every 30 minutes from 08:00 to 19:30.



## Updated code: adding timestamps to image filenames

In its original configuration KiloCam does not have the capability to add a timestamp to the images captured on the ESP32-Cam board, due to the fact that the ESP32-Cam board does not have continuous power or an onboard real-time clock with battery backup. As a result, users typically need to rename their images with the appropriate date, or adjust the image metadata using software such as Adobe Lightroom, based on the programmed sampling interval and the image number - this can get understandably tricky when capturing images at unique intervals instead of 24 hours a day.

**To resolve this issue I am providing additional code and this guide for installing it on your ESP32-Cam board. No changes are needed to the code uploaded to your KiloCam PCB.** As an overall concept this new code takes the last known image number and estimates the timestamp of the current photo based on the sampling interval and sleep schedule that a user provides in a text file on the microSD card. Before we begin I want to emphasize three things:

1. This code is still in development and there may be bugs that arise in its use.
2. If you do not provide the appropriate text file, or reset this file prior to deployment, the timestamps in your image filenames will be incorrect.
3. The code will not properly assign timestamps if your initial wakeup value is not the same as your dawn value. If you set your dawn value to be 08, set your initial wakeup time to also be 08 (see below).

**Step 1:** Download the new code [from this link](#). When opening the Arduino sketch file you will be prompted to create a subfolder for this sketch to live in, click accept and the Arduino IDE will do this automatically. **Please note that you need to already have installed all libraries required for a typical KiloCam build as [instructed in this video](#). Make sure you have installed the “Time” library as listed in the video description. No extra libraries are needed beyond those listed.**

**Step 2:** Upload the new code to your ESP32-Cam. You do not need to make any adjustments to the new ESP32-Cam code file, just upload it in the same way as you did with the original ESP32-Cam code as instructed to in the [build guide video](#). Ensure that you have the correct board selected in the Arduino IDE and the correct wiring before uploading - this is noted in the build guide video as well.

**Step 3:** Open the code you uploaded to your **KiloCam PCB** and write down the user-specified values for the **initial wakeup time, photo capture interval, dawn, and dusk**. For the purpose of this demonstration let's assume these values are:

- 2021/01/01 08:00:00 - An initial wakeup time of 08:00 on January 1 2021
- 900 - A photo capture interval of 900 seconds (or 15 minutes)
- 08 - A dawn value of 08, meaning photos will not be taken before 08:00
- 20 - A dusk value of 20, meaning photos will not be taken after the last possible interval within the hour of 20:00.

**Step 4:** Plug your ESP32-Cam's microSD card into your laptop. Add a text file with the filename "settings" (it should read as settings.txt). On a Mac you can do this using the TextEdit application, and on a Windows PC you can do this using the Notepad application. Open this file and add the following information to it as a single line with values separated by commas:

YYYY/MM/DD HH:MM:SS, INTERVAL, DAWN, DUSK, IMAGENUMBER, DEMO

Where these values correspond to :

- YYYY/MM/DD HH:MM:SS - the initial wakeup time and date of your KiloCam PCB (see above). The date is usually the next day if you are programming your board after the initial wakeup hour value.
- INTERVAL - the photo capture interval used when programming your KiloCam PCB
- DAWN - the dawn value used when programming your KiloCam PCB
- DUSK - the dusk value used when programming your KiloCam PCB
- IMAGENUMBER - set this to 0, this is the number of images taken so far in this deployment. The ESP32-Cam will increment this number each time it wakes up. This number should be reset to 0 prior to each deployment.
- DEMO - set this to 0, this is incremented to one after the demo cycle that occurs after startup. This number should be reset to 0 prior to each deployment.

**For our example values above, the text file should look like this prior to saving:**

2021/01/01 08:00:00, 900, 08, 20, 0, 0

Save the “settings.txt” file, properly eject your microSD card and insert it into your ESP32-Cam board. The unit is now ready to deploy and will include the projected timestamp in the filename of each image captured along with an image number. When deploying multiple KiloCam units with the same settings you can just copy and past this settings file to each unit’s microSD card.

#### A few important notes:

- You will need to update this file before each KiloCam deployment in order for the ESP32-Cam to properly estimate the time an image was taken. The values in the ESP32-Cam’s settings file should always match those used in your deployed KiloCam code and the values for IMAGENUMBER and DEMO should always be reset to 0. **There should be no other files besides the settings.txt file on a microSD card prior to deployment.**
- If no settings file is provided, the ESP32-Cam board will not capture data.
- If you are capturing data 24 hours a day, use the values 0 for Dawn and 23 for Dusk in both your settings.txt file and in your KiloCam code. I improperly instructed some to use 0 for Dawn and 24 for dusk previously. If you are capturing data around the clock it might be easier to use the original, reliable ESP32-Cam code as calculating your own timestamps should be straightforward.
- Always run a bench test to ensure that your KiloCam is operating correctly. The KiloCam PCBs are robust and tested prior to shipping, but ESP32-Cam boards have about a 20% dead-on-arrival rate. Running a bench test ensures that your ESP32-Cam board is able to properly capture images, open the microSD card file system and write to it.
- Always make sure your memory cards are formatted as FAT32. Not Ex-FAT or FAT16. Any windows computer should be able to format your cards to FAT32.